Security Report — PathFinder AI

Table of Contents

1. System Overview & Architecture
2. Authentication, MFA & RBAC Design
3. Application/API Security Controls
4. Network & Deployment Security
5. Logging & Alerting Design
6. DevSecOps Security Pipeline
7. Security Testing Performed & Results
8. Vulnerabilities Identified & Remediated
9. Known Limitations & Production Recommendations

1. System Overview & Architecture

1.1 Application Purpose

PathFinder AI is an intelligent career guidance application that uses machine learning models to help students (Class 10 through Graduate) predict optimal career paths, compare salary projections, and connect with successful alumni. The application supports multiple user roles: students at various stages, parents, viewers, and administrators.

1.2 Technology Stack

Layer	Technology	Justification
Frontend	React 18 + Vite + TypeScript	SPA architecture; React's DOM escaping prevents reflected XSS
Backend API	FastAPI (Python 3.10)	Async support, built-in Pydantic validation, OpenAPI docs auto-disabled in production
Database	MongoDB 6.0 (via Motor async driver)	Document-oriented storage suitable for flexible user profiles and audit logs
Reverse Proxy / WAF	Nginx 1.25 with custom WAF rules	TLS termination, rate limiting, request filtering
Containerization	Docker + Docker Compose	Reproducible deployment, network isolation, least-privilege containers
CI/CD	GitHub Actions	Automated security scanning on every push

1.3 Architecture Overview

The application follows a **multi-tier architecture** with three isolated Docker networks:



Key design decision: Only ports 80 and 443 are exposed to the host. The backend (port 8000) and MongoDB (port 27017) are **never** directly reachable from the internet.

1.4 Communication Paths

Source	Destination	Protocol	Purpose
Internet	WAF	HTTPS (443)	All user traffic
Internet	WAF	HTTP (80)	Redirected to HTTPS via 301
WAF	Frontend	HTTP (internal)	Serve React SPA
WAF	Backend	HTTP (internal)	Proxy `/api/*` requests
Backend	MongoDB	TCP 27017 (internal)	Data persistence and audit logs

2. Authentication, MFA & RBAC Design

2.1 Password Security

- **Algorithm:** Argon2id via `passlib[argon2]`
- **Rationale:** Argon2id is the winner of the Password Hashing Competition and resists both GPU-based and side-channel attacks
- **Minimum length:** 12 characters (enforced by Pydantic `Field(min_length=12)`)
- **Timing-attack defense:** When a login attempt uses a non-existent email, a `dummy_verify()` function hashes a throwaway string to prevent timing-based user enumeration

2.2 JWT Access Tokens (RS256)

- **Algorithm:** RS256 (asymmetric RSA-2048 signing)
- **Lifetime:** 10 minutes (below the required 15-minute maximum)
- **Key management:** RSA key pair generated at deployment time via `scripts/gen-secrets.sh` and mounted as Docker secrets at `/app/secrets/private.pem` and `/app/secrets/public.pem`
- **Claims included:**

Claim	Purpose
`sub`	User ID (UUID)
`role`	User's current role
`type`	Token type (`access`)
`jti`	Unique token ID (16 random bytes, hex)
`exp`	Expiration time (10 min from issue)
`nbf`	Not-before time (10 sec grace for clock skew)
`iat`	Issued-at time
`iss`	Issuer: `pathfinder-api`
`aud`	Audience: `pathfinder-client`

- **Verification:** Every protected endpoint verifies `iss`, `aud`, `exp`, `type`, and `role` on the server side before granting access

2.3 Refresh Token Handling

- **Format:** 32 random bytes (hex), stored as SHA-256 hash in MongoDB
- **Delivery:** `HttpOnly`, `Secure`, `SameSite=Strict` cookie scoped to `/api/auth`
- **Lifetime:** 7 days
- **Rotation:** On every `/api/auth/refresh` call, the old token is revoked and a new one issued (within the same `family_id`)

- **Reuse detection:** If a revoked refresh token is presented, **all tokens in the family are revoked** and a `REFRESH_TOKEN_REUSE_DETECTED` critical security event is logged. This mitigates token theft scenarios.

2.4 Multi-Factor Authentication (MFA)

- **Type:** TOTP (Time-based One-Time Password) via `pyotp`
- **Flow:**
  1. Admin calls `POST /api/auth/mfa/setup` → receives TOTP secret + QR provisioning URI
  2. Admin scans QR code in authenticator app (Google Authenticator, Authy, etc.)
  3. On subsequent logins, after password verification, API returns `mfa_required: true` with a short-lived (5-minute) MFA-pending token
  4. Admin submits TOTP code via `POST /api/auth/mfa/verify` → receives access + refresh tokens
- **Secret storage:** MFA secrets are encrypted at rest using Fernet symmetric encryption (key from environment variable)
- **Enforcement:** Required for all `admin` accounts

2.5 Role-Based Access Control (RBAC)

The application implements 7 distinct roles:

Role	Description	Access Level
`admin`	System administrator	Full access: audit logs, alerts, user management, config, exports
`class10`	Class 10 student	ML predictions, lifecycle data, senior matching
`class12`	Class 12 student	ML predictions, lifecycle data, senior matching
`college`	College student	ML predictions, salary projection, senior matching
`graduate`	Graduate	ML predictions, salary projection
`parent`	Parent/guardian	Read-only access to linked child's data
`viewer`	Read-only viewer	Basic read access to public endpoints

Enforcement mechanism: The `require_roles()` dependency function:

1. Extracts and validates the JWT from the `Authorization: Bearer <token>` header
2. Verifies the RS256 signature, expiration, issuer, and audience
3. Checks that the token's `role` claim is in the list of `allowed_roles`
4. Logs `FORBIDDEN_ACCESS` event if role check fails (403)
5. Logs `UNAUTHORIZED_ACCESS` event if token is invalid/expired (401)

2.6 Account Lockout

- **Threshold:** 5 consecutive failed login attempts
- **Lockout duration:** 15 minutes

- **Behavior:** The `locked_until` timestamp is checked on every login attempt. Even correct credentials are rejected during the lockout period (HTTP 423)
- **Reset:** Counter resets to 0 on successful login

2.7 Admin Bootstrapping

- **No default/shared accounts exist.** The first admin is created via a secure CLI command run inside the Docker container:

```
docker exec -it pathfinder_backend python app/cli.py create-admin
```

- The CLI prompts for email and password interactively (password via `getpass` — not echoed to terminal)
- The admin creation event is logged in the tamper-evident audit chain

---

3. Application/API Security Controls

3.1 Input Validation

All API inputs are validated using **Pydantic v2** strict schemas:

- `extra = "forbid"` — rejects any unexpected fields in request bodies
- `EmailStr` — validates email format (prevents operator injection)
- `Field(min_length=..., max_length=..., ge=..., le=..., pattern=...)` — enforces bounds on all numeric and string fields
- Role field uses regex pattern: `^(class10|class12|college|graduate|parent|admin|viewer)$`

Example: The `UserLogin` schema:

```
class UserLogin(BaseModel):
    email: EmailStr
    password: str = Field(..., min_length=1)
    class Config:
        extra = "forbid"  # Blocks {"$gt": ""} injection
```

3.2 Injection Prevention

- **NoSQL Injection:** MongoDB operators like `{"$gt": ""}` are rejected because Pydantic's `EmailStr` type validates the email as a string, not an object. The `extra = "forbid"` setting blocks any additional fields (e.g., `$where`, `$regex`)
- **Parameterized queries:** All database operations use Motor's typed query interface via Python dictionaries — no string concatenation or raw query construction
- **Pickle deserialization:** ML model files are loaded via `secure_load_pickle()` which verifies SHA-256 checksums against a manifest before calling `pickle.load()`

3.3 Cross-Site Scripting (XSS) Protection

- **Output encoding:** React automatically escapes all values rendered in JSX, preventing reflected and stored XSS
- **Content Security Policy:** Set at both the WAF level (Nginx) and the application level (FastAPI middleware):

```
default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https://f
```

3.4 Security Headers

All responses include the following security headers (set by both the Nginx WAF and FastAPI middleware for defense-in-depth):

Header	Value	Purpose
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains; preload`	Forces HTTPS for 1 year
`X-Frame-Options`	`DENY`	Prevents clickjacking via iframes
`X-Content-Type-Options`	`nosniff`	Prevents MIME-type sniffing
`X-XSS-Protection`	`1; mode=block`	Legacy browser XSS filter
`Referrer-Policy`	`strict-origin-when-cross-origin`	Controls referrer leakage
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	Restricts browser features
`Content-Security-Policy`	See above	Controls resource loading
`Cache-Control`	`no-store`	Prevents caching of API responses

3.5 CORS Policy

CORS is restricted to specific origins:

```
allow_origins=["https://localhost", "http://localhost"]
allow_methods=["GET", "POST", "PUT", "DELETE"]
allow_credentials=True
```

Wildcard origins (`*`) were identified as a vulnerability in the baseline assessment and have been remediated.

3.6 Error Handling

- **User-facing:** A global exception handler catches all unhandled exceptions and returns a generic message: `{"error": "An internal server error occurred.", "request_id": "<uuid>"}`

- **Server-side:** Full exception details are logged via Python's `logging` module with the request ID for correlation
- **Login errors:** Both "email not found" and "wrong password" return the same message: `"Invalid email or password"` to prevent user enumeration

3.7 WAF-Level Protections (Nginx)

The WAF configuration (`nginx/waf.conf`) includes pattern-based blocking for:

- **SQL injection patterns:** `union select` , `insert into` , `drop table` , `concat(` , `sleep(` , `benchmark(`
- **XSS patterns:** `<script>` , `javascript:` , `vbscript:` , `onXXX=` event handlers
- **Path traversal:** `../` , `%2e%2e` , `etc/passwd` , `proc/self`
- **Vulnerable endpoints:** `wp-admin` , `phpmyadmin` , `.env` , `.git` , `.svn`
- **Server version hiding:** `server_tokens off`

---

4. Network & Deployment Security

4.1 Docker Network Isolation

Three Docker networks enforce strict segmentation:

Network	Type	Connected Containers	Purpose
`pathfinder_edge`	External	WAF only	Internet-facing entry point
`pathfinder_app`	Internal (`internal: true`)	WAF, Frontend, Backend	Application layer — no direct internet access
`pathfinder_db`	Internal (`internal: true`)	Backend, MongoDB	Database layer — only backend can reach the DB

Key isolation: MongoDB cannot be reached from `pathfinder_edge` or the internet. The frontend cannot reach MongoDB. Only the backend container bridges `pathfinder_app` and `pathfinder_db` .

4.2 Exposed Ports

Port	Service	Notes
80	Nginx WAF	Redirects to 443
443	Nginx WAF	TLS-terminated HTTPS

Port 8000 (backend) and port 27017 (MongoDB) are **not** mapped to the host.

4.3 TLS Configuration

- **Protocols:** TLSv1.2 and TLSv1.3 only (TLSv1.0 and TLSv1.1 are disabled)

- **Cipher suites:** ECDHE-based AEAD ciphers only (AES-128-GCM, AES-256-GCM)
- **Certificate:** Self-signed RSA-2048 for local development (generated via `scripts/gen-certs.sh`)
- **Session caching:** `shared:SSL:10m` with 1-day timeout for performance

4.4 Container Hardening

All containers apply the following security controls:

Control	Implementation
Non-root user	Backend runs as `appuser`; Nginx runs as nginx user
Read-only filesystem	`read_only: true` with necessary `tmpfs` mounts
Capability drops	`cap_drop: [ALL]` with only required capabilities re-added
No privilege escalation	`security_opt: [no-new-privileges:true]`
Memory limits	WAF: 256MB, Frontend: 256MB, Backend: 1024MB, MongoDB: 1024MB

4.5 Rate Limiting

Zone	Rate	Burst	Scope
`login_limit`	5 req/sec	3	`/api/auth/login` only
`api_limit`	20 req/sec	10	All `/api/*` endpoints
`general_limit`	30 req/sec	20	Frontend static assets

4.6 Firewall Configuration

A `scripts/firewall-ufw.sh` script is provided for host-level firewall hardening:

- Default: deny incoming, allow outgoing
- Allow: ports 80 and 443
- Allow: SSH from admin CIDR only
- Deny: ports 27017 and 8000 explicitly

---

5. Logging & Alerting Design

5.1 Security Audit Logger

The `SecurityAuditLogger` class creates **tamper-evident, hash-chained** audit logs:

Hash chain mechanism:

1. Each log event includes a `prev_hash` field pointing to the previous event's hash
2. The current event's hash is computed as: `SHA-256(prev_hash + canonical_json(record))`
3. The first event in the chain uses `prev_hash = "GENESIS"`

4. Any modification to a historical log entry breaks the hash chain, making tampering detectable

Storage: Events are written to both:

- **MongoDB** (`security_events` collection) — for querying and alerting
- **Local JSONL file** (`/app/logs/security.jsonl`) — for backup and external SIEM forwarding

5.2 Logged Security Events

Event Type	Severity	Trigger
`LOGIN_SUCCESS`	info	Successful password + (optional MFA) authentication
`LOGIN_FAILURE`	medium	Wrong password or non-existent email
`ACCOUNT_LOCKED`	high	5th consecutive failed login attempt
`LOGOUT`	info	User-initiated session termination
`SIGNUP_FAILURE`	low	Attempt to register with existing email
`USER_CREATED`	info	New user registration
`UNAUTHORIZED_ACCESS`	medium	Invalid/expired JWT presented
`FORBIDDEN_ACCESS`	medium	Valid JWT but insufficient role permissions
`MFA_ENROLLED`	info	User enables TOTP MFA
`MFA_SUCCESS`	info	Correct TOTP code submitted
`MFA_FAILURE`	medium	Incorrect TOTP code submitted
`TOKEN_REFRESHED`	info	Successful token rotation
`REFRESH_TOKEN_REUSE_DETECTED`	critical	Previously revoked refresh token reused (possible theft)
`ROLE_CHANGE`	high	Admin changes a user's role
`CONFIG_CHANGE`	high	Admin modifies application configuration
`DATA_EXPORT`	high	Admin exports user data or audit logs
`ADMIN_ACTION`	info	Any admin panel action (view logs, list users, etc.)

5.3 Event Schema

Every event contains:

```
{
  "event_id": "UUID",
  "timestamp": "ISO 8601 UTC",
  "event_type": "LOGIN_FAILURE",
  "severity": "medium",
  "outcome": "failure",
  "actor_user_id": "UUID or null",
  "actor_role": "class10",
  "target": "resource or user ID",
  "source_ip": "172.18.0.1",
  "user_agent": "Mozilla/5.0...",
  "request_id": "UUID",
  "method": "POST",
  "path": "/api/auth/login",
  "status_code": 401,
  "details": {"reason": "Invalid credentials"},
  "prev_hash": "abc123...",
  "hash": "def456..."
}
```

5.4 Alert Engine

An asynchronous background task (`AlertEngine`) runs every 60 seconds, evaluating 5 detection rules:

Rule ID	Severity	Trigger Condition	Purpose
`rule_brute_force_account`	High	≥5 `LOGIN_FAILURE` events for same account in 5 min	Detect credential guessing
`rule_brute_force_ip`	High	≥10 `LOGIN_FAILURE` events from same IP in 5 min	Detect distributed attacks
`rule_privilege_escalation`	Medium	≥5 `UNAUTHORIZED_ACCESS` / `FORBIDDEN_ACCESS` from same IP in 10 min	Detect privilege probing
`rule_token_theft`	Critical	Any `REFRESH_TOKEN_REUSE_DETECTED` event in 5 min	Detect stolen sessions
`rule_excessive_export`	High	≥3 `DATA_EXPORT` events by same user in 10 min	Detect data exfiltration

Alerts are stored in MongoDB (`security_alerts` collection) with deduplication: repeated triggers of the same rule increment the `count` and update `last_seen` rather than creating duplicate alerts.

---

6. DevSecOps Security Pipeline

6.1 Pipeline Overview

A GitHub Actions workflow (`.github/workflows/security-pipeline.yml`) runs on every push and pull request to `main` , plus nightly at 02:00 UTC.

6.2 Pipeline Stages

Stage	Tool	What It Checks	Blocking Threshold
SAST	Semgrep	Source code patterns: OWASP Top 10, Python, TypeScript, React, JWT, Secrets	`--severity ERROR --error` (blocks on ERROR findings)
SCA	pip-audit + npm audit	Known vulnerabilities in Python and Node.js dependencies	`--audit-level=high` (blocks on HIGH+)
Secret Scanning	Gitleaks	Hardcoded secrets, API keys, passwords in git history	Any finding blocks
Container Scanning	Trivy	OS and library vulnerabilities in Docker images	`exit-code: 1` on HIGH/CRITICAL with `ignore-unfixed: true`
DAST	OWASP ZAP	Runtime vulnerabilities against running application	Baseline scan with alerting

6.3 Security Gate

A final `security-gate` job depends on all 5 scanning jobs. It only passes if every upstream job succeeds — ensuring that any HIGH/CRITICAL finding blocks the deployment.

6.4 Scan Configuration

- **Semgrep rules:** `p/owasp-top-ten`, `p/python`, `p/typescript`, `p/react`, `p/jwt`, `p/secrets`
- **Trivy:** Scans both frontend and backend Docker images; ignores unfixed CVEs
- **Gitleaks:** Configured via `.gitleaks.toml`; full git history scan (`fetch-depth: 0`)
- **OWASP ZAP:** Baseline scan against `http://localhost:80` with custom rules

---

7. Security Testing Performed & Results

Test 1: RBAC Enforcement — Viewer Cannot Access Admin Endpoint

Field	Details
Attack Scenario	A user with `viewer` role calls `GET /api/admin/audit` using their valid JWT
Impact Without Control	Any authenticated user could view all audit logs, user data, and security alerts
Security Control	`require_roles(["admin"])` dependency validates JWT role claim server-side
How Tested	Created a `viewer` user via `/api/auth/signup`, obtained JWT via `/api/auth/login`, then called `/api/admin/audit` with that token
Result	HTTP 403 Forbidden with `FORBIDDEN_ACCESS` event logged including the user's ID, role, and IP
Evidence	`tests/security/test_security.py::test_viewer_cannot_access_admin_endpoint`

Test 2: Brute Force → Account Lockout

Field	Details
Attack Scenario	Attacker sends 5+ login attempts with wrong passwords for the same account
Impact Without Control	Attacker could brute-force weak passwords indefinitely
Security Control	`failed_logins` counter increments per attempt; at 5 failures, `locked_until` is set to `now + 15 minutes`; returns HTTP 423
How Tested	Created a test user, sent 5 requests with wrong password, then tried the correct password
Result	Attempts 1-5 return 401; attempt 6 (even with correct password) returns 423 "Account is locked"
Evidence	`tests/security/test_security.py::test_account_lockout_after_failed_logins`

Test 3: NoSQL Injection Prevention

Field	Details
Attack Scenario	Attacker sends `{"email": {"$gt": ""}, "password": "anything"}` to bypass authentication
Impact Without Control	MongoDB interprets `{"$gt": ""}` as "match any email greater than empty string," returning the first user and bypassing password verification
Security Control	Pydantic `EmailStr` type rejects non-string values with HTTP 422; `extra = "forbid"` blocks additional MongoDB operators
How Tested	Sent POST to `/api/auth/login` with operator objects in both email and password fields
Result	HTTP 422 Unprocessable Entity — Pydantic rejects the request before it reaches MongoDB
Evidence	`tests/security/test_security.py::test_nosql_injection_in_login_email`, `test_nosql_injection_in_login_password`, `test_extra_fields_rejected`

Test 4: XSS Payload Handling

Field	Details
Attack Scenario	Attacker registers with `name = "<script>alert('XSS')</script>"` to inject stored XSS
Impact Without Control	Script executes when admin views user list, stealing session tokens
Security Control	React escapes all JSX output; CSP blocks inline scripts; WAF blocks `<script>` in URL parameters
How Tested	Submitted signup with XSS payload in `name` and `careerGoal` fields
Result	String is stored as-is in MongoDB but rendered harmlessly by React's DOM escaping; CSP `script-src 'self'` blocks any inline execution attempt
Evidence	`tests/security/test_security.py::test_xss_in_signup_name`, `test_xss_in_career_goal`

Test 5: Forged/Expired JWT Rejection

Field	Details
Attack Scenario	Attacker crafts a JWT with `{"role": "admin"}` signed with a different key
Impact Without Control	Attacker gains admin access without valid credentials
Security Control	RS256 asymmetric verification ensures only tokens signed with the server's private key are accepted; `iss` and `aud` claims are validated
How Tested	Sent requests with: (a) a hand-crafted HS256 JWT, (b) a modified RS256 JWT with wrong signature, (c) a token missing "Bearer " prefix
Result	All three return HTTP 401 with `UNAUTHORIZED_ACCESS` logged
Evidence	`tests/security/test_security.py::test_forged_jwt_rejected`, `test_expired_token_rejected`, `test_missing_bearer_prefix`

Additional Tests Performed

Test	Result
Out-of-range ML input (marks=999)	422 — Pydantic `le=100` constraint blocks it
Negative marks (marks=-50)	422 — Pydantic `ge=0` constraint blocks it
Weak password (< 12 chars)	422 — Pydantic `min_length=12` blocks it
Duplicate email signup	400 — Server checks uniqueness before insert
Generic error messages	Login failures return identical messages regardless of failure reason
Security headers present	HSTS, X-Frame-Options, X-Content-Type-Options, CSP all verified
Health endpoint public	`/api/health` returns 200 without authentication

---

8. Vulnerabilities Identified & Remediated

Baseline Assessment Findings

ID	Vulnerability	Severity	Status
VULN-1	Client-side authentication with plaintext passwords in `localStorage`	Critical	**Remediated** — All auth logic moved to FastAPI backend with Argon2id
VULN-2	Hardcoded admin credentials (`ADMIN_EMAIL` , `ADMIN_PASSWORD`) in source code	Critical	**Remediated** — Removed all hardcoded credentials; admin created via secure CLI
VULN-3	No authentication or authorization on backend API endpoints	Critical	**Remediated** — All endpoints protected with `require_roles()` JWT validation
VULN-4	Wildcard CORS (`allow_origins=["*"]` + `allow_credentials=True`)	High	**Remediated** — CORS restricted to `https://localhost`
VULN-5	No rate limiting, brute-force protection, or MFA	High	**Remediated** — Account lockout, Nginx rate limiting, TOTP MFA implemented
VULN-6	Unsafe `pickle.load()` without integrity verification	High	**Remediated** — SHA-256 manifest verification before deserialization
VULN-7	Verbose error messages exposing stack traces and DB schema	Medium	**Remediated** — Global exception handler returns generic messages
VULN-8	Sensitive files committed to repository (`.pkl` , `dev_output.txt` , credentials in README)	Medium	**Remediated** — `.gitignore` updated; sensitive files removed

DevSecOps Scan Findings

Findings from the CI/CD pipeline (Semgrep, pip-audit, Trivy, Gitleaks) are captured in the pipeline run artifacts. Any HIGH/CRITICAL findings block the build until remediated.

9. Known Limitations & Production Recommendations

9.1 Current Limitations

Limitation	Mitigation
Self-signed TLS certificate	Acceptable for local dev; not for production
WAF uses pattern matching, not ModSecurity CRS engine	Covers OWASP Top 10 patterns but may miss novel attacks
No centralized log aggregation	Logs are in MongoDB + local JSONL files
No real-time alerting notifications (email/SMS/webhook)	Alert engine detects anomalies but only stores them in DB
Docs disabled in production but no API gateway authentication	API endpoints are still reachable if WAF is bypassed
MFA secrets encrypted with Fernet (symmetric)	Single key compromise exposes all MFA secrets

9.2 Network Segmentation & DMZ Placement

In production, the architecture should use proper network segmentation:

- **DMZ (Public Subnet):** WAF/Load Balancer with DDoS protection (e.g., AWS Shield, Cloudflare)
- **Application Subnet (Private):** Backend API servers with no direct internet access; outbound traffic only via NAT gateway
- **Data Subnet (Private, Isolated):** MongoDB with no internet access; accessible only from the application subnet
- **Management Subnet:** Jump box / bastion host for administrative SSH access with key-based authentication
- **Network ACLs and Security Groups** should enforce least-privilege communication between subnets

9.3 Restricted Administrative Access

- **VPN or bastion host:** Admin access should require VPN (e.g., WireGuard, AWS Client VPN) or SSH through a hardened bastion
- **IP allowlisting:** The `ADMIN_ALLOWED_CIDRS` feature should be tightened to specific admin IPs
- **Session recording:** All admin SSH sessions should be recorded (e.g., AWS Session Manager)
- **Privileged Access Management (PAM):** Consider CyberArk, HashiCorp Vault, or AWS IAM Identity Center for admin credential management

9.4 Centralized/Tamper-Evident Logging & Retention

- **Log forwarding:** Deploy Filebeat or Fluentd as a sidecar container to forward JSONL logs to a SIEM (Splunk, ELK Stack, or AWS CloudWatch)
- **Immutable storage:** Store audit logs in append-only storage (e.g., AWS S3 with Object Lock, or a blockchain-based audit trail)

- **Retention policy:** Maintain security logs for a minimum of 1 year (per most compliance frameworks); implement automated archival to cold storage
- **Log integrity:** The hash-chain mechanism should be supplemented with digital signatures for non-repudiation

9.5 Secure Key & Secret Management

- **Vault solution:** Replace file-based secrets with HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault
- **Key rotation:** Implement automated rotation of:
  - RSA JWT signing keys (with key ID (`kid`) in JWT header for seamless rotation)
  - Fernet MFA encryption key
  - MongoDB credentials
  - TLS certificates (via Let's Encrypt + cert-manager)
- **HSM:** For highly sensitive deployments, use Hardware Security Modules for JWT signing keys

9.6 Backup & Recovery

- **Database backups:** Automated daily MongoDB backups using `mongodump` with encryption at rest (AES-256)
- **Offsite storage:** Replicate backups to a geographically separate region
- **Recovery testing:** Quarterly restore tests to verify backup integrity
- **RPO/RTO:** Define Recovery Point Objective (< 1 hour) and Recovery Time Objective (< 4 hours)
- **Disaster recovery:** MongoDB replica set for high availability; application deployed across multiple availability zones

9.7 Security Assessment Before Production

- **Penetration test:** Engage a third-party security firm for comprehensive penetration testing (OWASP Testing Guide methodology)
- **Code audit:** Independent security code review focusing on authentication, authorization, and data handling
- **Compliance assessment:** Verify against relevant frameworks (OWASP ASVS Level 2, SOC 2 Type II, or GDPR/POPIA depending on jurisdiction)
- **Threat modeling:** Conduct STRIDE threat modeling for all data flows
- **Vulnerability disclosure program:** Establish a responsible disclosure policy

9.8 Additional Production Hardening

- **Web Application Firewall:** Replace custom pattern matching with a managed WAF (AWS WAF, Cloudflare WAF) or deploy ModSecurity with the OWASP Core Rule Set
- **API Gateway:** Add an API gateway (Kong, AWS API Gateway) for centralized authentication, throttling, and observability
- **Content Delivery Network:** Use a CDN for the React frontend to reduce load and add geographic DDoS protection

- **Monitoring:** Deploy Prometheus + Grafana for application metrics; PagerDuty or OpsGenie for incident alerting
- **JWT denylist:** Implement a Redis-backed JWT denylist for immediate token revocation (currently relying on short expiry only)